PSTAT 134 Final Project

Eric Livshiz, Jake Vurpillat, Dylan Crookes, Tanveer Singh, Samuel Erlikhman

2026-06-08

Introduction

We live in the age of streaming services. In the U.S., Netflix, Prime Video, and Disney+ are the most popular, with many households often having many subscriptions. (https://www.statista.com/chart/25382/most-used-video-streaming-platforms/?srsltid=AfmBOoox1XeLW3YlbDt9wzhbtbIxPH0Cxz3AqZt9FTke_DGyZl5rTXjR). While it's nice to have options, users may start to feel choice overload with the amount of content to choose from; having too many options can result in no choice. So, a user may spend time scrolling endlessly, with no result (at least we do). So, we thought of a question which many have been thought of before; can we use data to recommend movies that better match a user's preferences?

This project utilizes recommender systems, as well as natural language processing (NLP). The recommender system is used for finding patterns within the data, and suggest similar films based on common features (i.e., a *Star Wars IV* query may return other *Star Wars* films, or *Indiana Jones* due to Harrison Ford starring in both, or *Jurassic Park*, due to John Williams composing the score of both films). Regarding NLP, a few of the features that define similarity and we intend to use are stored in text format (key plot words as an example). We can of course parse the list and analyze the words individually, but we want to preserve structural representations (think of all the famous movie quotes which will lose meaning otherwise!). We also aim to build an NLP-based chat interface, which allows the user to input a natural query (i.e., "recommend me a movie similar to *Top Gun*"), and receive a useful output.

Goal

We want to answer the following question: "Can we accurately recommend movies given film metadata, such as genre, keywords, cast, and other relevant features?", using recommender systems and NLP, as well as to create a user interface to interact with our model

Data

We are lucky that we are not the only ones who care about statistics and films! There exists plenty of datasets regarding films, and we have chosen to use The Movies Dataset from Kaggle, as it contains data for ~45,000 films (we use "film" and "movie" interchangeably throughout this write-up, and want to point this out here to avoid any confusion, let's hope our model believes there is no difference in these terms as well). The data is collected from MovieLens, a massive dataset which we will not be using, as it contains 33 million ratings and 86,000 movies, and is not reliant for research, and the details, credits, and keywords are taken from TMDB API. The dataset contains structured and unstructured data, and we will focus on the following files included:

- `movies_metadata.csv` - overall movie information, i.e., title and genre
- `keywords.csv` - quite useful, as it contains plot-related keywords which are stored as JSON objects
- `credits.csv` - self-explanatory; the information about cast and crew
- `links.csv` - mapping between the MovieLens, TMDB, and IMDB identifiers

The dataset includes two more CSVs, `links_small.csv` and `ratings_small.csv`, which are a subset of 9,000 movies - we want to work with the larger dataset to be more accurate, and we are not sure how the subset was selected (we are not sure if it is random, the first 9,000, or what algorithm what used to select the subset). It is important to note, that this dataset was last updated in 2017, meaning our dataset is missing thousands of recent movies. Looking at the above example, a natural similar film to *Top Gun* would be *Top Gun: Maverick*, which came out in 2022, but our system would fail to recommend it.

Methodology

We'd like to introduce how we will build our recommendation and NLP systems.

Recommendation System This project stems from a content-based recommender systems, meaning that movies are recommended based on their similarity to other movies. This "similarity" is taken from film characteristics, i.e., genre and cast. We will implement this idea by making each movie a feature vector based on its attributes, we will use a similarity measure to be able to compare the movies, compute similarity score, and output the best n options. This can be achieved by using cosine similarity, where $cos(\theta) = \frac{A \cdot B}{||A||||B||}$. We measure the angle between two feature vectors, where A and B are two movies. The numerator is the dot product, meaning the overlap of the films' features, and the denominator normalizes given the length of vector. The resulting value is from -1 to 1; -1 means that the films are absolute opposites, whereas 1 means that the movies are almost identical (think *Mission Impossible* films). Cosine similarity is quite useful in our context as it focuses on feature patterns, rather than the scale.

NLP Natural language processing is used in this project to understand text-based movie features, as well as for user interaction. As previously mentioned, the keywords and cast lists are stored in a text format and are not easily interpreted by a computer. One way to deal with this data is to find the most useful text-based features (genre and keyword, versus studio or release date). We merge these features into a text representation for each movie (which is referred to as a bag of words), and convert the text to numerical representation using the Term Frequency-Inverse Document Frequency (TF-IDF) method. This process assigns weights to words based on how important it is to the entire dataset $TF - IDF(t, d) = TF(t, d) \cdot log(\frac{N}{DF(t)})$. We can then feed natural sentences to a chatbot, have it process the query, and formulate it into a proper query to the recommender system. Let's go ahead and get started with the data cleaning.

Data Cleaning

Let's start by introducing the datasets provided:

```
keywords <- read_csv("data/movies/keywords.csv")
links <- read_csv("data/movies/links.csv")
ratings <- read_csv("data/movies/ratings.csv")
movies <- read_csv("data/movies/movies_metadata.csv")
credits <- read_csv("data/movies/credits.csv")

dataset_summary <- tibble(
  dataset = c("credits", "keywords", "links", "ratings", "movies_metadata"),
  observations = c(
    nrow(keywords),
    nrow(links),
    nrow(ratings),
    nrow(movies),
    nrow(credits)
  ),
```

```r
  variables = c(
    ncol(keywords),
    ncol(links),
    ncol(ratings),
    ncol(movies),
    ncol(credits)
  )
) %>% kable()

column_summary <- tibble(
  dataset = c("keywords", "links", "ratings", "movies_metadata", "credits"),
  columns = c(
    paste(colnames(keywords), collapse = ", "),
    paste(colnames(links), collapse = ", "),
    paste(colnames(ratings), collapse = ", "),
    paste(colnames(movies), collapse = ", "),
    paste(colnames(credits), collapse = ", ")
  )
)

column_summary %>% kable()
```

dataset	columns
keywords	id, keywords
links	movieId, imdbId, tmdbId
ratings	userId, movieId, rating, timestamp
movies_metadata	adult, belongs_to_collection, budget, genres, homepage, id, imdb_id, original_language, original_title, overview, popularity, poster_path, production_companies, production_countries, release_date, revenue, runtime, spoken_languages, status, tagline, title, video, vote_average, vote_count
credits	cast, crew, id